RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop

HIGH LEVEL DESIGN DOCUMENT

Internship Project | Batch: adv-genai-feb-2026
Submitted by: Ms. Keerthana G
Intern ID: IN126046402

Table of Contents

1. System Overview

1.1 Problem Definition

Traditional customer support systems rely heavily on human agents who must manually search through large volumes of documentation, knowledge bases, and product manuals to answer customer queries. This approach is slow, inconsistent, and expensive to scale. As support volumes grow, maintaining quality while reducing response times becomes increasingly difficult.

This project addresses these limitations by building a Retrieval-Augmented Generation (RAG) based Customer Support Assistant. The system automatically ingests PDF knowledge base documents, understands the content, and answers user queries intelligently using the power of Large Language Models (LLMs) guided by relevant retrieved context.

1.2 Scope of the System

The system is designed to:

- Accept any PDF document as a knowledge base input
- Automatically parse, chunk, and embed the document content
- Store embeddings in a persistent vector database (ChromaDB)
- Respond to user queries using context-aware LLM generation (Groq / LLaMA 3.1)
- Execute a structured graph-based workflow using LangGraph
- Route queries — either to an LLM answer node or to a HITL escalation node
- Provide a clean Streamlit-based web interface for user interaction

The system does NOT currently support real-time email/ticket systems, multi-user authentication, or production deployment. These are scoped for future phases.

2. Architecture Diagram

The diagram below represents the high-level system architecture including all major layers from the user interface to LLM response generation:

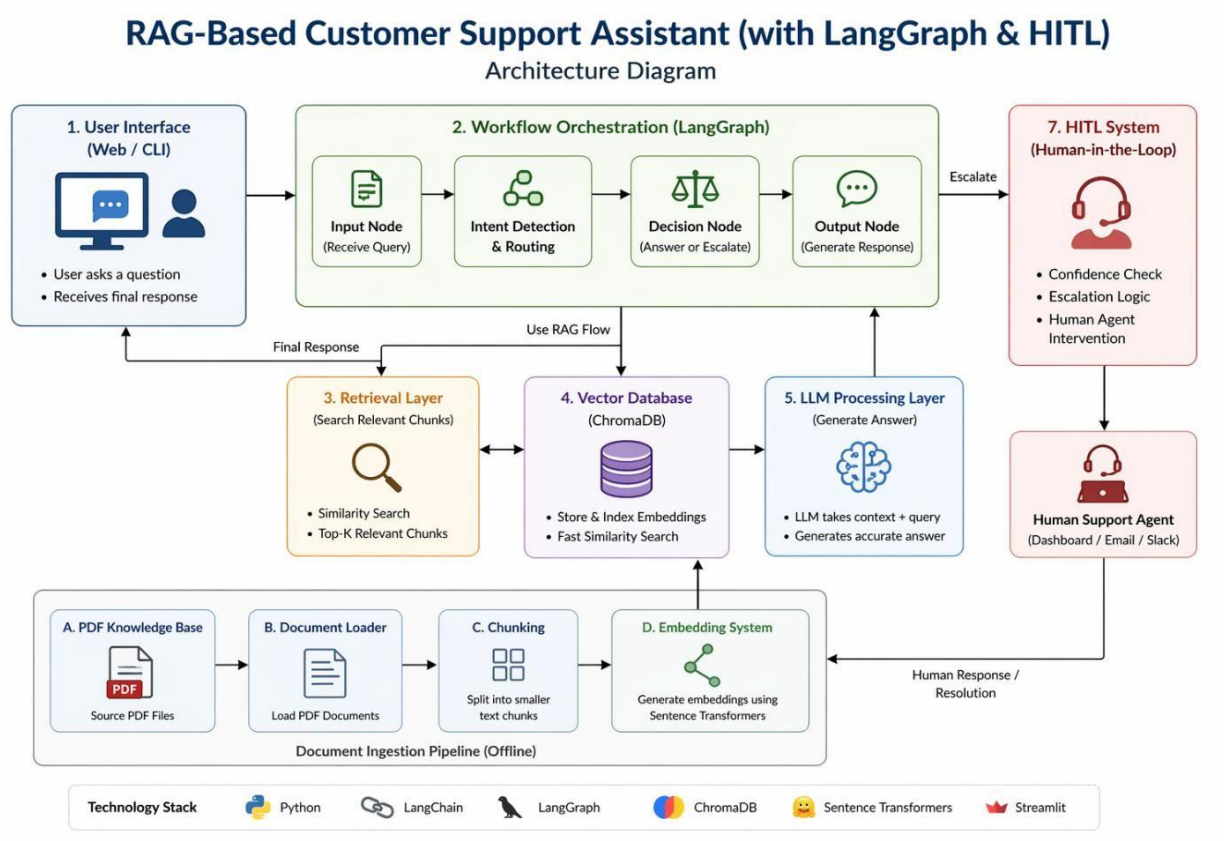


Fig 2.1: Architecture Diagram

3. Component Description

Each component in the system serves a distinct responsibility. Below is a detailed description of each module:

3.1 Document Loader

Module	pipeline.py → load_pdf()
Library	pypdf (PdfReader)
Input	PDF file path (temp.pdf)
Output	Raw extracted text string
Responsibility	Reads each page of the uploaded PDF and concatenates all extractable text into a single string for downstream processing.

3.2 Chunking Module

Module	pipeline.py → chunk_text()
Strategy	Fixed-size sliding window chunking
Chunk Size	500 characters
Overlap	50 characters (maintains context continuity between chunks)
Output	List of text chunks (List[str])
Rationale	Prevents context loss at boundaries. Overlap ensures semantic relationships across chunk edges are preserved.

3.3 Embedding Model

Module	pipeline.py → create_vector_store()
Current Implementation	FakeEmbeddings (size=384) — for development/testing
Production Upgrade	all-MiniLM-L6-v2 via sentence-transformers (768-dim)
Purpose	Converts text chunks into dense numerical vectors for semantic search
Note	FakeEmbeddings generates random vectors; production must use real embeddings.

3.4 Vector Store

Module	pipeline.py → create_vector_store()
Technology	ChromaDB via langchain_community.vectorstores.Chroma
Mode	In-memory (current); supports persistent disk storage
Purpose	Stores chunk embeddings and enables fast nearest-neighbor search
Why ChromaDB	Lightweight, Python-native, no separate server needed, OOTB LangChain integration

3.5 Retriever

Module	pipeline.py → get_retriever()
Method	vectordb.as_retriever(search_kwargs={'k': 3})
Search Type	Approximate Nearest Neighbor (ANN) similarity search
Top-K	Returns top 3 most relevant chunks per query
Output	List of Document objects with page_content attribute

3.6 LLM (Groq API)

Module	pipeline.py → generate_answer()
Provider	Groq API (groq Python SDK)
Model	llama-3.1-8b-instant
Temperature	0.2 (low — for factual, consistent answers)
Max Tokens	500
Prompt Strategy	System-injected rules + context + question format

3.7 Graph Workflow Engine (LangGraph)

Module	graph.py → build_graph()
Framework	LangGraph — StateGraph
Nodes	process, answer, hitl
Entry Point	process node
Finish Points	answer node, hitl node
State Object	Python dict with keys: question, context, route, response
Purpose	Orchestrates the query lifecycle with conditional branching logic

3.8 Routing Layer

Location	graph.py → builder.add_conditional_edges()
Logic	If context length < 50 chars → route = 'HITL', else route = 'ANSWER'
Trigger	Short/empty context indicates retrieval failure (no matching chunks)
Output	Routes graph execution to either answer node or hitl node

3.9 HITL Module

Module	graph.py → hitl() node
Current Implementation	Returns static escalation message to user
Future Implementation	Ticket creation in Zendesk / Freshdesk / email dispatch
Trigger Condition	Insufficient context retrieved (< 50 characters)
Purpose	Ensures no query is silently dropped — human agent is looped in

4. Data Flow

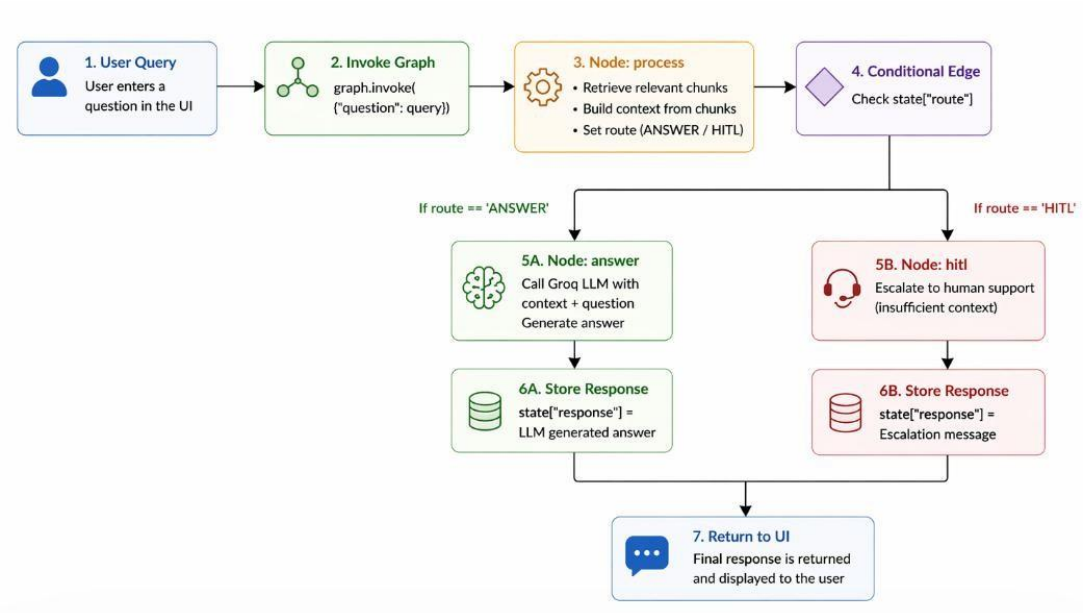
4.1 PDF to Answer — Complete Data Flow

The following describes the end-to-end journey of a document and a user query through the system:

Step	Action	Module	Output
1	User uploads PDF via Streamlit	app.py	PDF file written to temp.pdf
2	PDF text extracted	pipeline.py → load_pdf()	Raw text string
3	Text split into chunks	pipeline.py → chunk_text()	List of 500-char chunks
4	Chunks embedded	pipeline.py → create_vector_store()	Vector representations
5	Vectors stored	ChromaDB	Indexed vector store
6	User submits query	app.py → text_input	Query string
7	Query embedded + retrieved	pipeline.py → get_retriever()	Top-3 relevant chunks
8	State initialized	graph.py → build_graph()	{'question': ..., 'context': ..., 'route': ...}
9	Conditional routing	graph.py → lambda routing fn	Route: ANSWER or HITL
10	LLM answer generated	pipeline.py → generate_answer()	Answer string
11	Response displayed	app.py → st.markdown()	Chat UI response box

4.2 Query Lifecycle

When a query is submitted, the graph is invoked with the initial state: {"question": query}. The process node retrieves relevant chunks, builds context, and sets the route. The conditional edge then directs execution to either the answer node (LLM call) or hitl node (escalation). The final response is stored in state["response"] and returned to the UI.



5. Technology Choices

5.1 Why ChromaDB?

Criterion	ChromaDB	FAISS	Pinecone
Setup Complexity	Very Low	Low	High (cloud account)
LangChain Integration	Native, built-in	Supported	Supported
Persistence	Optional (in-memory/disk)	File-based	Cloud-native
Cost	Free, open-source	Free	Paid tiers
Use Case Fit	Prototyping + Production	Research	Large-scale production
Python-Native	Yes	Yes	No (REST API)

ChromaDB was selected because it requires zero external server setup, integrates natively with LangChain, and supports both in-memory (fast prototyping) and persistent storage modes.

5.2 Why LangGraph?

- Enables building stateful, cyclical agent workflows — unlike simple LangChain chains
- Provides explicit node-edge graph semantics, making control logic visible and modifiable
- Native support for conditional routing — critical for HITL decision branching
- State is an explicit Python dict — easy to inspect, debug, and extend
- Designed for production-grade agentic systems by LangChain team

5.3 LLM Choice — Groq + LLaMA 3.1

Factor	Decision
Provider	Groq API — ultra-low latency inference
Model	llama-3.1-8b-instant — fast, capable, free tier
Temperature	0.2 — minimal hallucination, factual output
Context Window	128K tokens — sufficient for retrieved context
Cost	Free tier supports prototyping at scale

5.4 Additional Tools

Tool	Version	Purpose
Streamlit	Latest	Web UI for PDF upload and Q&A chat
pypdf	Latest	PDF text extraction
LangChain Community	Latest	ChromaDB integration, Retriever API
LangGraph	Latest	Graph-based workflow engine
Python	3.11+	Core runtime

6. Scalability Considerations

6.1 Handling Large Documents

The current chunking strategy (500 char, 50 overlap) is suitable for documents up to ~200 pages. For larger documents:

- Increase chunk size to 1000–2000 characters with semantic boundary awareness
- Use recursive character text splitter (LangChain) for paragraph-aware chunking
- Enable ChromaDB persistent mode to avoid re-ingestion on each session
- Implement async PDF loading with a progress indicator in the UI

6.2 Handling Increasing Query Load

- Deploy on multi-worker Streamlit or switch to FastAPI backend for concurrency
- Cache frequent query-context pairs using Redis or LRU in-memory cache
- Batch embed queries using sentence-transformers GPU acceleration
- Rate-limit Groq API calls per user session to prevent quota exhaustion

6.3 Latency Concerns

Bottleneck	Current	Solution
PDF Ingestion	Synchronous — blocks UI	Async background task + progress bar
Embedding Generation	FakeEmbeddings (fast but fake)	Sentence-transformers GPU (production)
Vector Retrieval	In-memory ChromaDB (fast)	Persistent ChromaDB with HNSW indexing
LLM Inference	Groq API ~200–500ms	Already optimized; cache for repeated queries
UI Rendering	Streamlit reruns full page	Use st.session_state efficiently

6.4 Multi-Tenant Scalability

For production deployment supporting multiple concurrent users, the system would require: separate ChromaDB collections per user session, JWT-based session isolation, a queue-based LLM request handler (e.g., Celery + Redis), and horizontal scaling of the Streamlit app via Docker containers with a load balancer.